

Documentation Addendum — Phase 1 Improvements

March 2026 | Supplements: DEVELOPER_GUIDE, DATABASE_GUIDE, OPERATIONS_GUIDE, TESTING_AND_DEPLOYMENT_GUIDE

This document records everything added or changed during the Phase 1 improvement sprint that is not yet reflected in the existing guides. When those guides are next revised, this addendum should be merged in and then deleted.

1. New Environment Variables

1.1 Frontend (frontend/.env.local / Vercel Dashboard)

The following variables are **new** and not listed in DEVELOPER_GUIDE.md §Environment Variables Reference.

Variable	Required	Description
SANITY_WEBHOOK_SECRET	Yes (prod)	HMAC-SHA256 secret for verifying incoming Sanity webhook payloads. Set the same value in Sanity → manage.sanity.io → your project → API → Webhooks.
LOOPS_API_KEY	Yes (prod)	API key from app.loops.so → Settings → API. Server-side only — never expose to the browser.
LOOPS_TEMPLATE_WELCOME	No	Loops transactional template ID for welcome emails. Leave blank to skip Loops and fall back to legacy HTML send.
LOOPS_TEMPLATE_DIGEST	No	Template ID for weekly digest emails.
LOOPS_TEMPLATE_PAYMENT_FAILED	No	Template ID for failed payment notifications.
LOOPS_TEMPLATE_TRIAL_ENDING	No	Template ID for trial-ending reminders.
LOOPS_TEMPLATE_SURVEY_RESULTS	No	Template ID for survey result delivery emails.
NEXT_PUBLIC_SENTRY_DSN	Yes (prod)	Sentry DSN for frontend error capture. Found in sentry.io → your project → Settings → Client Keys.
SENTRY_ORG	Yes (prod)	Your Sentry organization slug (used for source map uploads during build).
SENTRY_PROJECT	Yes (prod)	Your Sentry project slug. Convention: htr-platform.
SENTRY_AUTH_TOKEN	Yes (prod)	Sentry auth token for source map uploads. Generate at sentry.io → Settings → Auth Tokens.

1.2 Backend (backend/.env / Railway Dashboard)

No new backend variables were added. The existing INGEST_SECRET variable now also protects the new GET /api/ingest/status endpoint (same Bearer token — see §3.2 below).

2. New Database Tables (Supabase Migrations)

The following tables were added via migrations and are **not yet documented in DATABASE_GUIDE.md**.

2.1 rag_query_log (Migration 015)

Logs every AI chat query for evaluation, content gap analysis, and latency monitoring.

```
CREATE TABLE rag_query_log (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id           UUID REFERENCES auth.users(id) ON DELETE SET NULL,  
  session_id        TEXT,  
  query             TEXT NOT NULL,  
  role              TEXT,                                -- user's tier at query time  
  model_used        TEXT,  
  retrieved_doc_ids TEXT[],                               -- IDs of retrieved rag_documents  
  retrieved_scores   FLOAT[],                             -- cosine similarity scores  
  response_preview   TEXT,                                -- first 500 chars of response  
  latency_ms        INT,  
  tokens_used        INT,  
  was_zero_result    BOOLEAN NOT NULL DEFAULT FALSE,  
  created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

RLS: Users can read only their own rows. Service role (backend) can read all.

Admin view: A pre-built aggregate view `rag_query_stats` is available:

```
SELECT * FROM rag_query_stats;  
-- Returns: day, total_queries, zero_result_queries, avg_latency_ms,  
p95_latency_ms, unique_users
```

This is the primary operational dashboard for RAG quality monitoring. Query it weekly to identify content gaps (high `zero_result_queries`) and latency regressions (`p95_latency_ms`).

2.2 bookmarks (Migration 016)

Stores per-user article bookmarks. Tied to Sanity document identity (`sanity_id` + `slug`).

```
CREATE TABLE bookmarks (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id           UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  sanity_id         TEXT NOT NULL,                        -- Sanity document _id  
  slug              TEXT NOT NULL,                        -- URL slug  
  title             TEXT NOT NULL,  
  pillar            TEXT,                                -- 'policy', 'economics', etc.  
  content_type       TEXT,                                -- 'policyAnalysis', 'caseStudy', etc.  
  note              TEXT CHECK (char_length(note) <= 2000),  
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  UNIQUE (user_id, sanity_id)  
);
```

RLS: Full access (SELECT/INSERT/UPDATE/DELETE) scoped to `user_id = auth.uid()`.

UI entry points:

- `BookmarkButton` component — rendered in the article action bar on every article page
- `/account/bookmarks` — the user's saved articles list

2.3 ingest_jobs (implicit, created by backend)

Tracks background index rebuild jobs triggered by `POST /api/ingest`. Written by the Python backend using the service role key.

```
-- Created inline by the backend; reproduce manually if needed:
CREATE TABLE ingest_jobs (
  id          UUID PRIMARY KEY,
  status      TEXT NOT NULL,          -- 'queued' | 'running' | 'completed'
  | 'failed'
  error_message TEXT,
  started_at   TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

No RLS is applied — this table is written and read exclusively by the backend service role.

2.4 role_change_log (recommended, not yet migrated)

Described in PLATFORM_IMPROVEMENT_PLAN.md §3.2. Not yet applied. When added, this table records every role change with old_role, new_role, changed_by, and reason for billing dispute audits. A trigger on user_roles auto-populates it.

3. New and Changed API Endpoints

These are additions and changes to the endpoint list in DEVELOPER_GUIDE.md.

3.1 POST /api/webhooks/sanity (Frontend — Next.js)

Receives Sanity content-change webhooks and triggers a RAG re-index.

- **Auth:** HMAC-SHA256 signature in sanity-webhook-signature header, verified against SANITY_WEBHOOK_SECRET
- **On success:** Calls POST <PYTHON_BACKEND_URL>/api/ingest with Authorization: Bearer \$INGEST_SECRET
- **Response:** 200 { received: true } or 401 if signature invalid

Setup in Sanity:

1. Go to manage.sanity.io → your project → API → Webhooks
2. Add webhook URL: https://your-domain.vercel.app/api/webhooks/sanity
3. Set the secret to match SANITY_WEBHOOK_SECRET in your Vercel env
4. Trigger on: Document create / update / delete

3.2 POST /api/ingest (Backend — Python/FastAPI) — Updated

Previously undocumented: this endpoint now returns 202 Accepted immediately and runs the index rebuild asynchronously in the background. It no longer blocks.

- **Auth:** Authorization: Bearer \$INGEST_SECRET (if INGEST_SECRET is set; open in dev)
- **Conflict handling:** Returns { status: "already_running" } if a job is in progress — safe to retry
- **Response:** { status: "accepted", job_id: "<uuid>", message: "..." }

3.3 GET /api/ingest/status (Backend — Python/FastAPI) — New

Poll the status of the most recent index rebuild job.

- **Auth:** Same Authorization: Bearer \$INGEST_SECRET header
- **Response (idle):** { status: "idle", message: "No ingest job has run since last restart." }
- **Response (active):** { job_id, status, queued_at, started_at?, completed_at?, error? }
- **Possible status values:** queued → running → completed | failed

Typical polling pattern:

```
# Trigger rebuild
curl -X POST https://your-backend.railway.app/api/ingest \
  -H "Authorization: Bearer $INGEST_SECRET"

# Poll until completed
curl https://your-backend.railway.app/api/ingest/status \
  -H "Authorization: Bearer $INGEST_SECRET"
```

4. AI Pipeline Changes

4.1 Tier-Aware Engine Routing

The `/api/chat` endpoint now routes to two different engines based on user tier. This is **not yet documented** in `DEVELOPER_GUIDE.md`.

Tier	Engine	Behavior
subscriber, student	ContextChatEngine	RAG-only. Retrieves context, streams answer. Faster and cheaper.
professional, advisory, admin	ReActAgent	Full agentic loop. LLM can call external tools (state data, research lab) before answering. Up to 5 reasoning iterations.

Tools available to the agentic pipeline are registered in `backend/services/tools.py` as `ALL_TOOLS`.

4.2 System Prompt Injection Guard

A prompt injection detector runs on every incoming message and `systemPrompt` field before the LLM sees them. Phrases like `"ignore previous instructions"`, `"repeat your system prompt"`, and 7 others are blocked. Blocked messages receive a canned deflection response — no error is surfaced to the user.

4.3 RAG Query Logging

Every chat request (except `user_id == "dev"`) is logged to `rag_query_log` after the response completes. Logged fields include the query, tier, model used, retrieved document IDs and scores, first 500 chars of the response, and latency. This is the primary input for Ragas evaluation (§6 below).

5. Error Monitoring (Sentry)

Sentry is now integrated in both frontend and backend. This is entirely new and not covered in any existing guide.

Frontend: Configured via `sentry.client.config.ts`, `sentry.server.config.ts`, and `sentry.edge.config.ts`. Source maps are uploaded at build time using `SENTRY_AUTH_TOKEN`. Errors are captured automatically by the Next.js SDK.

Backend: Configure via `SENTRY_DSN` environment variable in Railway. Add `sentry-sdk[fastapi]` to `requirements.txt` and call `sentry_sdk.init()` in `main.py` (not yet done — see §7 below).

Triage workflow:

1. Go to `sentry.io` → your project → Issues
2. Filter by `environment:production`
3. Click any issue to see the full stack trace, user context, and request payload
4. The `user_id` is attached to each event (set via `sentry_sdk.set_user()` in the auth middleware)

6. RAG Evaluation with Ragas

A golden test set and evaluation runner now exist at `backend/eval/evaluate_rag.py`. This is **entirely new** and not documented in `TESTING_AND_DEPLOYMENT_GUIDE.md`.

Running an evaluation

```
# Install eval dependencies (one-time)
pip install "ragas~=0.2" "datasets~=2.0" pandas

# Run from the backend directory
cd backend
python -m eval.evaluate_rag
```

The script:

- 1. Loads the golden Q&A dataset (defined inline in the file)
- 2. Runs each question through the live RAG pipeline
- 3. Scores four metrics via Ragas: **faithfulness**, **answer_relevancy**, **context_precision**, **context_recall**
- 4. Prints a summary table and saves results to `eval/results_<timestamp>.csv`

Interpreting results

Metric	Target	Meaning if low
faithfulness	> 0.85	LLM is hallucinating — not sticking to retrieved context
answer_relevancy	> 0.80	Answers are off-topic or vague
context_precision	> 0.75	Retrieved chunks are not relevant to the question
context_recall	> 0.70	Retrieved chunks are missing information needed to answer

Expanding the test set

The golden dataset is defined in `GOLDEN_DATASET` inside `evaluate_rag.py`. It currently has ~12 Q&A pairs. **Target: 50+ pairs** covering all five pillars (Policy, Economics, Technology, Clinical, Equity) before treating eval results as statistically meaningful.

Add entries in this format:

```
{
  "question": "...",
  "ground_truth": "...",    # expected factual answer
  "pillar": "Policy",       # for filtering/segmentation
}
```

When to run

- Before and after every content ingest to detect regressions
- After changing chunking strategy, retrieval top-k, or re-ranker settings
- Monthly as a baseline health check

7. Known Gaps Not Yet Addressed

These items were identified but **not implemented** in this phase. Record here to avoid losing them.

7.1 Backend Sentry SDK Not Initialized

`NEXT_PUBLIC_SENTRY_DSN` is wired into the frontend, but the Python backend does not yet call `sentry_sdk.init()`. To complete Sentry coverage:

```
# Add to requirements.txt:
# sentry-sdk[fastapi]~=2.0

# Add to backend/main.py (before app creation):
import sentry_sdk
sentry_sdk.init(
    dsn=os.getenv("SENTRY_DSN"),
    traces_sample_rate=0.1,
    environment=os.getenv("RAILWAY_ENVIRONMENT", "development"),
)
```

7.2 Ragas Dependencies Not in `requirements.txt`

`ragas`, `datasets`, and `pandas` are needed to run `evaluate_rag.py` but are intentionally excluded from `requirements.txt` (they are eval-only, not production dependencies). Install them locally before running:

```
pip install "ragas~=0.2" "datasets~=2.0" pandas
```

Consider adding a separate `requirements-eval.txt` to make this self-documenting.

7.3 `role_change_log` Migration Not Yet Applied

Migration `017_role_change_log.sql` has not been written or applied. The schema is documented in `PLATFORM_IMPROVEMENT_PLAN.md §3.2` and in §2.4 above. Apply before going to production if billing dispute auditing is required.

7.4 Loops Email Template IDs Not Configured

`LOOPS_TEMPLATE_*` variables are blank in `.env.production.example`. Email flows (welcome, digest, payment failed, trial ending) will silently no-op until these are set. Configure templates in the Loops.so template editor, then add the IDs to Vercel project environment variables.

7.5 `vercel.json` Domain Placeholders

Two values in `vercel.json` must be updated before go-live:

- CORS `Access-Control-Allow-Origin` header: replace `https://your-domain.vercel.app` with your actual Vercel URL
- The `PYTHON_BACKEND_URL` env var in Vercel dashboard (not in `vercel.json`) must be set to the Railway backend URL

End of addendum. Merge into respective guides during next documentation pass.